



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**PROYECTO "APLICACIÓN MÓVIL COLABORATIVA PARA
MEJORAR LA EFECTIVIDAD EN LA ASISTENCIA DE MASCOTAS
PERDIDAS CON PARTICIPACIÓN DE VOLUNTARIOS EN LA
CIUDAD DE TACNA, 2026"**

Curso: Construcción de software II

Docente: RICARDO EDUARDO VALCARCEL ALVARADO

Integrantes:

Chite Quispe Brian Danilo (2021070015)

Anampa Pancca David Jordan (2022074268)

Tacna – Perú

2026

ÍNDICE GENERAL

1. Modelo Entidad / Relación	3
1.1 Diseño Lógico	3
1.2 Diseño Físico	4
2. Diccionario de Datos — Colecciones Firestore	5
2.1 Colección: usuarios	5
2.2 Colección: reportes	8
2.3 Colección: avistamientos	11
2.4 Colección: notificaciones	13
2.5 Colección: chats	15
2.6 Colección: comentarios	16
2.7 Colección: patacoin_transacciones	18
3. Procedimientos Almacenados — Firestore Rules	19
4. Lenguaje de Definición de Datos (DDL)	21
DDL — Estructura de Colecciones en Modelos Dart	21
DDL — Índices Compuestos en Firestore	22
5. Lenguaje de Manipulación de Datos (DML)	22
DML — Operaciones CRUD para la Colección reportes	22
DML — Operaciones para la Colección avistamientos	23
DML — Operaciones para la Colección notificaciones	24

1. Modelo Entidad / Relación

1.1 Diseño Lógico

El diseño lógico del sistema SOS Mascota representa la estructura conceptual de la base de datos y define cómo se organizan y relacionan los datos a nivel conceptual sin considerar las limitaciones físicas de implementación. Para el sistema se ha optado por un modelo de datos **NoSQL** utilizando **Firestore**, lo que permite una mayor flexibilidad en el manejo de datos no estructurados propios de aplicaciones colaborativas de rescate animal en tiempo real.

Las entidades principales se organizan en colecciones que contienen documentos, estableciéndose relaciones mediante referencias de identificadores únicos (`uid`, `reporteId`, `autorId`) en lugar de claves foráneas tradicionales de bases de datos relacionales. Este enfoque resulta particularmente adecuado para el manejo de datos con variaciones de estructura (por ejemplo, los reportes pueden tener o no vector de embeddings según si el usuario subió imagen), permitiendo escalado horizontal eficiente y rendimiento optimizado para operaciones de lectura y escritura frecuentes en escenarios de rescate.

Entidades Principales Identificadas:

Colección	Descripción
usuarios	Registro completo de voluntarios, dueños de mascotas y administradores con validación RENIEC
reportes	Publicaciones de mascotas perdidas, encontradas o en adopción con geolocalización
avistamientos	Registro de avistamientos geolocalizados reportados por voluntarios con cotejo biométrico
notificaciones	Historial de notificaciones push y alertas comunitarias por usuario
chats	Salas de mensajería privada entre voluntario y dueño de mascota
comentarios	Comentarios comunitarios en reportes con hilos de respuesta y reacciones

Colección	Descripción
patacoin_transacciones	Registro de acreditaciones y uso de PataCoins del sistema de recompensas

1.2 Diseño Físico

El diseño físico constituye la implementación concreta del modelo lógico en Firebase Cloud Firestore, considerando todos los aspectos técnicos y de rendimiento. Los identificadores de documentos son generados automáticamente por Firestore garantizando unicidad y consistencia. Los campos de tipo temporal se manejan como objetos `Timestamp` nativos de Firestore para asegurar precisión en las operaciones de geolocalización y coincidencia.

Características Técnicas del Diseño Físico:

Característica	Implementación
Base de Datos	Firebase Cloud Firestore (NoSQL documental)
Identificadores	Generación automática UUID por Firestore
Manejo Temporal	<code>Timestamp</code> nativo de Firestore con conversión UTC
Índices	Compuestos para consultas multi-campo frecuentes (<code>estado + fechaPublicacion</code> , <code>tipo + distrito</code>)
Validaciones	Implementadas en la capa de aplicación (Flutter/Dart — ViewModels)
Seguridad	Reglas de seguridad por colección configuradas en <code>firestore.rules</code>
Escalabilidad	Automática, gestionada por Firebase Google Cloud
Vectores de IA	<code>Arrays List<double></code> de 1280 dimensiones (<code>embeddings MobileNet</code>) almacenados como campos Firestore

Fuente: Elaboración propia del equipo de trabajo.

2. Diccionario de Datos — Colecciones Firestore

2.1 Colección: usuarios

Descripción: Registro completo de todos los usuarios del sistema incluyendo autenticación, datos personales validados por RENIEC, configuración de rol y monedero de recompensas PataCoins.

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
1	uid	String (auto- generado)	UUID	NO	PK	Identificador único universal generado automáticamente por Firebase Authentication. Actúa como clave primaria del documento en Firestore.
2	nombre	String	max 100	NO	—	Nombre completo del usuario auto-completado desde la API RENIEC al ingresar el DNI. Usado para identificación en el feed y los chats.
3	apellidos	String	max 100	NO	—	Apellidos completos del usuario obtenidos de la API RENIEC (miapi.cloud).
4	dni	String	8 dígitos	NO	UNIQUE	Número de DNI peruano validado contra la API RENIEC en el proceso de registro. Garantiza identidad real del voluntario.
5	email	String	max 255	NO	UNIQUE	Dirección de correo electrónico usada como

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
					UE	identificador de autenticación en Firebase Auth. Verificada mediante email de confirmación.
6	password	String (hash SHA-256)	64 hex	NO	—	Hash seguro de la contraseña del usuario generado mediante SHA-256 con salting. Nunca se almacena en texto plano.
7	rol	String (enum)	max 20	NO	—	Nivel de acceso del usuario en el sistema: "usuario" (voluntario / dueño de mascota) o "admin" (moderador con acceso al panel administrativo).
8	telefono	String	max 20	SÍ	—	Número de teléfono en formato internacional. Campo opcional para contacto directo.
9	fotoPerfil	String (URL)	max 500	SÍ	—	URL de Cloud Storage de la fotografía de perfil del usuario. Vacio si el usuario no ha subido foto.
10	tokenFCM	String	max 200	SÍ	—	Token de Firebase Cloud Messaging del dispositivo activo del usuario. Se actualiza en cada inicio de sesión para garantizar recepción de

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
						notificaciones push.
11	pataCoins	Integer	—	NO	—	Saldo actual de PataCoins del usuario en el monedero de recompensas. Valor inicial: 0. Incrementa 10 puntos por avistamiento acreditado.
12	fechaCreacion	Timestamp	—	NO	—	Marca temporal exacta de creación de la cuenta en Firebase Authentication.
13	fechaActualizacion	Timestamp	—	SÍ	—	Marca temporal de la última modificación del perfil del usuario.
14	emailVerificado	Boolean	—	NO	—	Indicador de que el correo electrónico ha sido verificado mediante el enlace enviado por Firebase Auth. Los usuarios no verificados no pueden publicar reportes.
15	activo	Boolean	—	NO	—	Estado de la cuenta: true = cuenta activa / false = cuenta suspendida por el administrador. El AuthWrapper expulsa en tiempo real a cuentas con activo == false.

Relaciones:

- 1:N con reportes — Un usuario puede publicar múltiples reportes de mascotas.
- 1:N con avistamientos — Un usuario puede registrar múltiples avistamientos.
- 1:N con notificaciones — Cada usuario acumula su historial de notificaciones.
- 1:N con patacoin_transacciones — Registro de movimientos del monedero.
- 1:N con chats — Un usuario puede participar en múltiples salas de chat.

Fuente: Elaboración propia del equipo de trabajo.

2.2 Colección: reportes

Descripción: Publicaciones de mascotas perdidas, encontradas o en adopción con geolocalización GPS, fotografías validadas por TFLite, y vector de embeddings para el algoritmo de cotejo biométrico.

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
1	id	String (auto-generado)	UUID	NO	PK	Identificador único del reporte generado automáticamente por Firestore.
2	usuarioId	String	UUID	NO	FK → usuarios .uid	UID del usuario dueño que publicó el reporte. Usado para notificarle cuando se detecta una coincidencia con un avistamiento.
3	tipoReporte	String (enum)	max 20	NO	—	Tipo de publicación: "perdido", "encontrado", "adopcion". Determina el ícono y color del marcador en el mapa (rojo, verde, naranja).
4	nombreMascota	String	max 100	NO	—	Nombre de la mascota registrado en el formulario wizard. Mostrado en el feed y en el cartel PDF de búsqueda.

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
5	especie	String (enum)	max 30	NO	—	Especie de la mascota: "perro", "gato", "ave", "otro". Usado para filtros en el feed.
6	raza	String	max 100	SÍ	—	Raza de la mascota si el dueño la conoce. Campo opcional.
7	color	String	max 100	NO	—	Descripción del color o combinación de colores del pelaje/plumaje de la mascota.
8	descripcion	String	max 1000	NO	—	Descripción detallada de la mascota: características distintivas, collares, microchip, comportamiento, zona habitual.
9	fotoUrl	String (URL)	max 500	NO	—	URL de Cloud Storage de la fotografía de la mascota. Validada previamente por el clasificador TFLite (confianza $\geq 60\%$).
10	embeddings	Array<Double>	1280 dims	SÍ	—	Vector de características de 1280 dimensiones extraído por el modelo MobileNet mediante TFLite. Usado para el algoritmo de similitud coseno en el cotejo biométrico.
11	latitud	Double	—	NO	—	Coordenada de latitud GPS del lugar donde se vio por

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
						última vez a la mascota o donde fue encontrada.
12	longitud	Double	—	NO	—	Coordenada de longitud GPS complementaria a la latitud. Ambas forman el punto geolocalizado visible en el mapa de Tacna.
13	distrito	String	max 100	SÍ	—	Nombre del distrito de Tacna donde ocurrió el avistamiento o pérdida. Extraído por geocodificación inversa.
14	estado	String (enum)	max 20	NO	—	Estado del reporte: "activo", "resuelto", "archivado". Los reportes resueltos suman a la tasa de éxito en el dashboard.
15	recompensa	Double	—	SÍ	—	Monto de recompensa en soles ofrecido por el dueño (opcional). Mostrado en el feed para motivar búsquedas.
16	fechaPublicacion	Timestamp	—	NO	—	Marca temporal de creación del reporte en Firestore.
17	fechaActualizacion	Timestamp	—	SÍ	—	Marca temporal de la última modificación del reporte.
18	contacto	String	max 200	NO	—	Información de contacto del dueño (teléfono o instrucción de chat). Incluida en el cartel PDF generado.

Relaciones:

- N:1 con usuarios — Múltiples reportes pertenecen a un único usuario dueño.
- 1:N con avistamientos — Un reporte puede recibir múltiples avistamientos de voluntarios.
- 1:N con comentarios — Un reporte puede acumular múltiples comentarios comunitarios.

Fuente: Elaboración propia del equipo de trabajo.

2.3 Colección: avistamientos

Descripción: Registro de avistamientos geolocalizados reportados por voluntarios. Incluye fotografía validada por TFLite, vector de embeddings para cotejo con reportes activos, y resultado del algoritmo de similitud coseno.

Nº	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
1	id	String (auto-generado)	UUID	NO	PK	Identificador único del avistamiento.
2	voluntarioId	String	UUID	NO	FK → usuarios .uid	UID del voluntario que registró el avistamiento. Recibe 10 PataCoins si el cotejo resulta en coincidencia confirmada.
3	fotoUrl	String (URL)	max 500	NO	—	URL de Cloud Storage de la fotografía tomada por el voluntario en el momento del avistamiento. Validada por TFLite.
4	embeddings	Array<Double>	1280 dims	SÍ	—	Vector de características de 1280 dimensiones extraído en el dispositivo del voluntario mediante inferencia TFLite.
5	latitud	Double	—	NO	—	Latitud GPS de la ubicación

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
						exacta del avistamiento. Capturada automáticamente del sensor GPS del dispositivo.
6	longitud	Double	—	NO	—	Longitud GPS del avistamiento. Junto con la latitud forma el punto geolocalizado.
7	descripcion	String	max 500	SÍ	—	Descripción opcional del voluntario sobre el avistamiento (color, comportamiento, dónde fue avistado exactamente).
8	reporteCoincidencia	String	UUID	SÍ	FK → reportes. id	ID del reporte con el que el algoritmo de similitud coseno detectó la mayor coincidencia (similaridad ≥ 0.5). Nulo si no se detectó coincidencia.
9	similitud	Double	0.0-1.0	SÍ	—	Valor de similitud coseno calculado entre el vector del avistamiento y el del reporte coincidente.
10	radioKm	Double	—	NO	—	Radio de búsqueda geográfica en km utilizado para el cotejo (valor constante: 9.0 km, configurable en AppConfig.radioKm).
11	pataCoinsAcreditados	Integer	—	NO	—	Puntos de PataCoins acreditados al voluntario por este avistamiento (valor: 10). Cero si el avistamiento fue rechazado.
12	fechaRegis	Timesta	—	NO	—	Marca temporal de creación del

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
	tro	mp				avistamiento en Firestore.
13	estado	String (enum)	max 20	NO	—	Estado del avistamiento: "pendiente", "confirmado", "descartado".

Relaciones:

- N:1 con usuarios — Múltiples avistamientos pertenecen a un voluntario.
- N:1 con reportes — Un avistamiento puede coincidir con un reporte activo.

Fuente: Elaboración propia del equipo de trabajo.

2.4 Colección: notificaciones

Descripción: Historial persistente de notificaciones push y alertas del sistema por usuario. Permite visualizar el historial aunque no se haya recibido el push (token FCM inválido o dispositivo apagado).

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
1	id	String (auto-generado)	UUID	NO	PK	Identificador único de la notificación.
2	usuarioId	String	UUID	NO	FK → usuarios.userId	UID del usuario destinatario de la notificación.
3	titulo	String	max 200	NO	—	Título de la notificación push (mostrado en la bandeja del sistema Android).
4	mensaje	String	max 500	NO	—	Cuerpo del mensaje de la notificación.
5	tipo	String	max 50	NO	—	Tipo de notificación: "coincidencia" (cotejo

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
		(enum)				biométrico positivo), "avistamiento" (nuevo avistamiento cercano), "sistema" (mensaje del administrador), "chat" (nuevo mensaje en chat privado).
6	referencia	String	UUID	SÍ	—	ID del documento relacionado (reporte, avistamiento o chat) para navegación al tocar la notificación.
7	leído	Boolean	—	NO	—	Indicador de si el usuario ha leído la notificación. false por defecto al crear. El badge de notificaciones se calcula con leído == false.
8	fecha	Timestamp	—	NO	—	Marca temporal de creación de la notificación en Firestore.

Relaciones:

- N:1 con usuarios — Múltiples notificaciones pertenecen a un usuario.
- Referencia opcional a reportes, avistamientos o chats mediante el campo referencia.

Fuente: Elaboración propia del equipo de trabajo.

2.5 Colección: chats

Descripción: Salas de mensajería privada en tiempo real entre el voluntario que registró un avistamiento y el dueño del reporte coincidente. Cada documento representa una sala de chat con subcolección de mensajes.

Nº	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
1	id	String	UUID	NO	PK	Identificador único de la sala de chat (generado como uid1_uid2 ordenados lexicográficamente).
2	participantes	Array<String>	2 UUIDs	NO	—	Array con los UUIDs de los dos participantes del chat (voluntario y dueño de mascota).
3	reporteId	String	UUID	SÍ	FK → id	ID del reporte que originó la sala de chat.
4	fechaCreacion	Timestamp	—	NO	—	Marca temporal de creación de la sala de chat.
5	ultimoMensaje	String	max 200	SÍ	—	Texto del último mensaje enviado. Para previsualización en la lista de chats.
6	fechaUltimoMensaje	Timestamp	—	SÍ	—	Marca temporal del último mensaje para ordenar los chats por actividad reciente.

Subcolección: chats/{chatId}/mensajes:

N°	Campo	Tipo	Descripción
1	id	String (auto-generado)	UUID del mensaje
2	autorId	String (FK → usuarios.uid)	UID del remitente del mensaje
3	contenido	String (max 2000)	Texto del mensaje
4	tipo	String (enum)	"texto" o "imagen"
5	fotoUrl	String (URL, nullable)	URL de Cloud Storage si el mensaje es una imagen
6	fecha	Timestamp	Marca temporal de envío del mensaje
7	leido	Boolean	true si el destinatario leyó el mensaje

Fuente: Elaboración propia del equipo de trabajo.

2.6 Colección: comentarios

Descripción: Comentarios comunitarios publicados en los reportes de mascotas. Incluye sistema de reacciones (Like/Dislike) y subcolección de respuestas en hilo.

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
1	id	String (auto-generado)	UUID	NO	PK	Identificador único del comentario.
2	reporteId	String	UUID	NO	FK → reporte s.id	ID del reporte al que pertenece el comentario.
3	autorId	String	UUID	NO	FK → usuari os.uid	UID del usuario que publicó el comentario.

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
4	autorNombre	String	max 100	NO	—	Nombre del autor en el momento de publicar. Almacenado para evitar consultas adicionales al renderizar.
5	contenido	String	max 2000	NO	—	Texto del comentario.
6	likes	Integer	—	NO	—	Contador de reacciones positivas. Actualizado con transacción atómica de Firestore.
7	dislikes	Integer	—	NO	—	Contador de reacciones negativas.
8	fecha	Timestamp	—	NO	—	Marca temporal de publicación del comentario.
9	editado	Boolean	—	NO	—	Indicador de si el comentario ha sido editado por el autor.

Subcolección: comentarios/{comentarioId}/replies: Mismo esquema que comentarios pero sin subcolección anidada adicional (máximo 1 nivel de anidamiento).

Fuente: Elaboración propia del equipo de trabajo.

2.7 Colección: `patacoin_transacciones`

Descripción: Registro inmutable de todas las acreditaciones y usos de PataCoins del otífie de gamificación. Proporciona trazabilidad completa del monedero de cada voluntario.

N°	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
1	id	String (auto- generado)	UUID	NO	PK	Identificador único de la transacción.
2	usuarioId	String	UUID	NO	FK → usuarios .uid	UID del voluntario al que corresponde la transacción.
3	tipo	String (enum)	max 20	NO	—	Tipo de movimiento: “credito” (ganancia de PataCoins) o “debito” (uso de PataCoins).
4	monto	Integer	—	NO	—	Cantidad de PataCoins involucrados en la transacción. Siempre otified; el tipo determina si es entrada o salida.
5	motivo	String (enum)	max 100	NO	—	Razón de la transacción: “avistamiento_acreditado”, “reporte_publicado”, “mascota_encontrada”, “canje_premio”.
6	referencia	String	UUID	SÍ	—	ID del documento relacionado (avistamiento o reporte) que otifi la transacción.
7	saldoAnterior	Integer	—	NO	—	Saldo de PataCoins antes de la transacción. Para

Nº	Campo	Tipo	Longitud	Permite Nulos	Clave	Descripción
						otified .
8	saldoPosterior	Integer	—	NO	—	Saldo de PataCoins después de la transacción.
9	fecha	Timestamp	—	NO	—	Marca temporal de la transacción.

Relaciones:

- N:1 con usuarios — Múltiples transacciones pertenecen a un voluntario.
- Referencia opcional a avistamientos o otifie mediante el campo referencia.

Fuente: Elaboración propia del equipo de trabajo.

3. Procedimientos Almacenados — Firestore Rules

Las reglas de seguridad de Firestore actúan como los procedimientos almacenados del otifie, controlando el acceso a nivel de colección y campo. Las reglas son gestionadas en el otifie `firestore.rules` del repositorio.

```
// Reglas de seguridad de Cloud Firestore — SOS Mascota
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Coleccion usuarios: solo el propio otifie o un admin puede leer/escribir
    match /usuarios/{uid} {
      allow read: if request.auth != null &&
        (request.auth.uid == uid || isAdmin());
      allow write: if request.auth != null &&
        (request.auth.uid == uid || isAdmin());
    }

    // Coleccion otifie: cualquier otifie autenticado puede leer, solo el dueño puede modificar
    match / otifie/{ otified} {
      allow read: if request.auth != null;
      allow create: if request.auth != null &&
```

```

        request.resource.data.usuarioId == request.auth.uid;
    allow update, delete: if request.auth != null &&
        (resource.data.usuarioId == request.auth.uid || isAdmin());
}

// Coleccion avistamientos: lectura abierta, escritura solo del voluntario
match /avistamientos/{avistamientoId} {
    allow read: if request.auth != null;
    allow create: if request.auth != null;
    allow update, delete: if request.auth != null &&
        (resource.data.voluntarioId == request.auth.uid || isAdmin());
}

// Coleccion notificaciones: solo el destinatario
match /notificaciones/{ notified} {
    allow read, write: if request.auth != null &&
        resource.data.usuarioId == request.auth.uid;
    allow create: if request.auth != null;
}

// Coleccion chats: solo participantes
match /chats/{chatId} {
    allow read, write: if request.auth != null &&
        request.auth.uid in resource.data.participantes;
}

// Funcion auxiliar para verificar rol de administrador
function isAdmin() {
    return get(/databases/${database}/documents/usuarios/${request.auth.uid}).data.rol == 'admin';
}
}
}

```

4. Lenguaje de Definición de Datos (DDL)

DDL — Estructura de Colecciones en Modelos Dart

Los modelos de datos del sistema SOS Mascota son clases Dart que implementan serialización/deserialización `fromMap()` / `toMap()` para interactuar con Firestore. Ejemplo de la colección reportes:

```
// lib/modelo/reporte_mascota.dart
class ReporteMascota {
  final String id;
  final String usuarioId;
  final String tipoReporte; // "perdido" | "encontrado" | "adopcion"
  final String nombreMascota;
  final String especie; // "perro" | "gato" | "ave" | "otro"
  final String? raza;
  final String color;
  final String descripcion;
  final String fotoUrl;
  final List<double>? embeddings; // Vector MobileNet de 1280 dims
  final double latitud;
  final double longitud;
  final String? distrito;
  final String estado; // "activo" | "resuelto" | "archivado"
  final double? recompensa;
  final Timestamp fechaPublicacion;
  final Timestamp? fechaActualizacion;
  final String contacto;

  factory ReporteMascota.fromMap(Map<String, dynamic> map) { ... }
  Map<String, dynamic> toMap() { ... }
}
```

DDL — Índices Compuestos en Firestore

Los índices compuestos optimizan las consultas más frecuentes del `reports` SOS Mascota:

Colección	Campos del Índice	Tipo	Propósito
reports	estado ASC, fechaPublicacion DESC	Compuesto	Cargar el feed de <code>reports</code> activos ordenados por más recientes
reports	especie ASC, estado ASC, fechaPublicacion DESC	Compuesto	Filtro del feed <code>reports</code> <code>ie</code> y estado
reports	usuarioId ASC, fechaPublicacion DESC	Compuesto	Pantalla “Mis Reportes” del usuario
avistamientos	voluntarioId ASC, fechaRegistro DESC	Compuesto	Historial de avistamientos del voluntario
notificaciones	usuarioId ASC, leído ASC, fecha DESC	Compuesto	Bandeja de notificaciones no leídas
comentarios	<code>reports</code> ASC, fecha ASC	Compuesto	Carga de comentarios de un reporte
patacoin_transacciones	usuarioId ASC, fecha DESC	Compuesto	Historial del monedero de PataCoins

5. Lenguaje de Manipulación de Datos (DML)

DML — Operaciones CRUD para la Colección `reports`

CREATE — Publicar nuevo reporte de mascota `reports`:

```
// lib/vistamodelo/ reports/reporte_vm.dart
Future<void> publicarReporte(ReporteMascota reporte) async {
  await FirebaseFirestore.instance
    .collection('reports')
    .doc(reporte.id)
    .set(reporte.toMap());
}
```

READ — Leer feed de reportes activos, ordenados por fecha, con paginación:

```
// Consulta Firestore con índice compuesto [estado ASC, fechaPublicacion DESC]
final snapshot = await FirebaseFirestore.instance
  .collection('reportes')
  .where('estado', isEqualTo: 'activo')
  .orderBy('fechaPublicacion', descending: true)
  .limit(AppConfig.limiteReportesPorPagina)
  .get();
```

READ — Cargar reportes activos en radio de 9 km para el cotejo de avistamiento:

```
// Pre-filtro geográfico: cuadro delimitador (bounding box) alrededor del voluntario
// Filtro fino: distancia euclidiana real calculada en avistamiento_vm.dart
final snapshot = await FirebaseFirestore.instance
  .collection('reportes')
  .where('estado', isEqualTo: 'activo')
  .get();
// Filtrado posterior por distancia Haversine < AppConfig.radioKm (9.0 km)
```

UPDATE — Cambiar estado del reporte a “resuelto”:

```
await FirebaseFirestore.instance
  .collection('reportes')
  .doc( reportes )
  .update({
    'estado': 'resuelto',
    'fechaActualizacion': Timestamp.now(),
  });
```

DELETE — Eliminar reporte (solo administrador o dueño):

```
await FirebaseFirestore.instance
  .collection('reportes')
  .doc( reportes )
  .delete();
```

DML — Operaciones para la Colección avistamientos

CREATE — Registrar avistamiento con acreditación de PataCoins:

```
// lib/vistamodelo/ reportes/avistamiento_vm.dart
// Transaccion atomica: crear avistamiento + acreditar PataCoins
```

```

await FirebaseFirestore.instance.runTransaction((tx) async {
  final avistamientoRef = FirebaseFirestore.instance
    .collection('avistamientos').doc();
  final usuarioRef = FirebaseFirestore.instance
    .collection('usuarios').doc(voluntarioId);

  tx.set(avistamientoRef, avistamiento.toMap());
  tx.update(usuarioRef, {
    'pataCoins': FieldValue.increment(AppConfig.pataCoinsPorAvistamiento),
  });
});

```

READ — Leer avistamientos recientes de un voluntario:

```

final snapshot = await FirebaseFirestore.instance
  .collection('avistamientos')
  .where('voluntarioId', isEqualTo: uid)
  .orderBy('fechaRegistro', descending: true)
  .limit(20)
  .get();

```

DML — Operaciones para la Colección `notificaciones`

CREATE — Registrar notificación de coincidencia en Firestore:

```

// lib/servicios/notificacion_servicio.dart — método _buildNotifData()
await FirebaseFirestore.instance
  .collection('notificaciones')
  .add({
    'usuarioId': duenioId,
    'titulo': titulo,
    'mensaje': mensaje,
    'tipo': 'coincidencia',
    'referencia': avistamientoId,
    'leido': false,
    'fecha': Timestamp.now(),
  });

```

UPDATE — Marcar notificación como leída:

```
await FirebaseFirestore.instance
  .collection('notificaciones')
  .doc(notified)
  .update({'leido': true});
```